

Week 7: Pipeline Runs, Reruns, Failures, and Recovery

Built for Cars, Deadly for Walkers: mining the roadway conditions behind Florida's pedestrian deaths, 2020-2024

Yelysei Shkirpan · Capstone Project, B.S. Data Science · Eastern Florida State College · October 5, 2026

Repository: <https://github.com/Sor1na/florida-pedestrian-patterns>

What this week adds. Weeks 4–6 designed the database and fixed the cleaning rules. This week the rules run. `pipeline/etl.py` takes the pinned raw files to the tables of the Week 5 schema through eleven stages and six validation gates. It logs every step with its run ID, time, and row counts, and it can be run again at any time without changing a correct database. `pipeline/run_scenarios.py` exercises it end to end and keeps the evidence: a first run, a rerun of the same inputs, two deliberate failures, and a process killed in the middle of the load followed by a restart. Each scenario ends with automatic checks that compare what happened with what should have happened.

Terms used below. "Rule 1–3" are the business rules of the Week 4 model ([README](#)): one crash per year and case number, at most one striking vehicle from the same crash, and a mandatory county. "Issue 1–5" are the five quality issues of the [Week 6 cleaning plan](#). The `v_dq_*` views (`sql/04_views.sql`) each return the rows that break one data-quality rule, so a good load leaves them empty. A **fingerprint** is an md5 hash over every row of the five data tables: two equal fingerprints mean byte-identical content.

Reproduce everything in this document. You need PostgreSQL 16 and Python 3.11 with `pip install -r requirements.txt`. Run from the repository root:

```
python pipeline/run_scenarios.py --synthetic      # synthetic inputs -> docs/evidence/week7_synthetic/
python pipeline/run_scenarios.py                  # the real files in data/raw/ -> docs/evidence/week7/
```

The scenarios run in their own database, `ped_safety_week7`, which is created if missing and rebuilt every time, so the project database `ped_safety` is never touched. The script exits with code 1 if any scenario check fails. The committed run: **19 checks passed, 0 failed**, on PostgreSQL 16.14 (the server version is the first line of `01_first_run.log`), code commit `b24aa7b` (on every run's first log line).

Which inputs the evidence comes from. FARS rows describe real deaths, and the proposal keeps person-level rows out of the public repository. The logs committed here therefore come from **synthetic input files** made by `tools/make_synthetic_raw.py`. These files copy the features of the real downloads that the pipeline depends on:

- the same zip and member names, in upper and lower case, with and without a folder;
- the same names for the columns they contain (the required columns plus a few others, for example 27 of the 80 accident columns; 7 of the 34 members);

- the cp1252 encoding of three 2020 members;
- the Census file in latin-1, with its state summary row;
- coded unknowns, filler coordinates, multi-victim crashes, and case numbers reused across years.

Every row in them is made up. The generator is deterministic, so the SHA-256 values in the logs can be reproduced.

For the synthetic evidence, the pinned file list and the reconciliation totals are the `sources.csv` and `expected_counts.csv` that the generator writes next to its files in `data/synthetic/clean/` (ignored by Git, reproducible). In those logs, the G5 label "expected (Week 6 profile / published)" therefore means the generator's totals (130 / 120 / 104 / 132 / 110 deaths), not the real Week 6 numbers.

The real files are pinned in `pipeline/sources.csv` with the checksums measured in Week 6 (`docs/profile/00_manifest_sha256.csv`). The reconciliation totals in `pipeline/expected_counts.csv` are the Week 6 profile numbers: 3,736 deaths in 3,690 crashes. The second command above runs the same scenarios on the real files and writes `docs/evidence/week7/`. That folder is not committed, because it can only be produced where the real files exist; its logs mask case numbers, CSV line numbers, and record values.

Scenario = run ID	What happens	Evidence	Section	Outcome
1	first run on an empty database	<code>01_first_run.log</code> , <code>runs/run_0001.log</code>	2	exit 0, succeeded
2	rerun with the same inputs	<code>02_rerun_same_inputs.log</code> , <code>runs/run_0002.log</code>	3	exit 0, identical fingerprint
3	failure A: tampered raw file	<code>03_failure_tampered_file.log</code> , <code>runs/run_0003.log</code>	4	exit 1 at G1, tables unchanged
4	failure B: four bad records	<code>04_failure_bad_records.log</code> , <code>runs/run_0004.log</code>	4	exit 1 at G4, 5 rejected records, tables unchanged
5	process killed inside the load	<code>05_interrupted_run.log</code> , <code>runs/run_0005.log</code>	4	exit 70, rolled back, run left <code>running</code>
6	restart: the same command	<code>06_restart_after_interrupt.log</code> , <code>runs/run_0006.log</code>	4	exit 0, run 5 closed, identical fingerprint
—	history of all six runs	<code>07_run_history.log</code>	5	all 19 scenario checks

1. Pipeline stages and dependencies

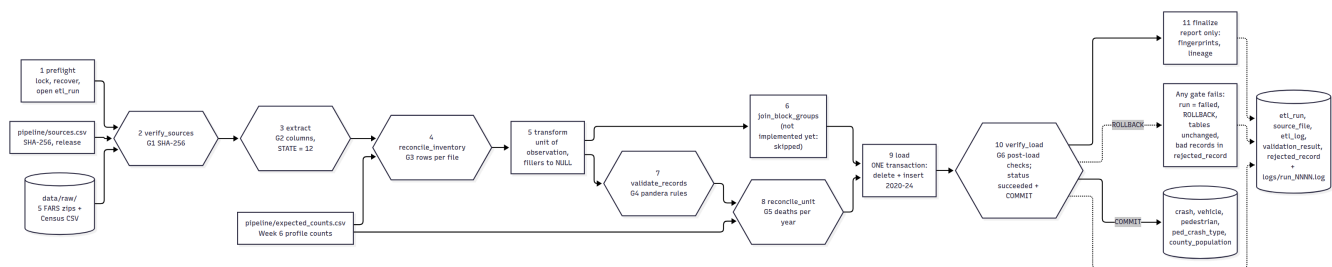


Diagram source: [docs/pipeline.mmd](#) (Mermaid; also [pipeline.svg](#)). The order is defined once, in STAGES in `pipeline/etl.py`. The runner refuses to start a stage whose dependencies have not finished. `python pipeline/etl.py` stages prints the same list from the code.

#	Stage	Runs after	Gate	What it does	A failure means
1	preflight	(start)	—	Connects and takes an advisory lock, so only one run can work at a time. Inserts the <code>etl_run</code> row (run ID, start time, git commit). Closes any other run left running by a dead process. Fingerprints the five data tables.	Exit 3 if another run holds the lock; exit 2 if the database is not built.
2	verify_sources	1	G1 integrity	Checks that every file in <code>sources.csv</code> is present and that its size and SHA-256 equal the pinned values. Writes one <code>source_file</code> row per file: source, year, release (Final or Annual Report File), URL, SHA-256, retrieval date.	A raw file is missing or changed. Nothing is read.
3	extract	2	G2 structure	Reads <code>accident</code> , <code>vehicle</code> , <code>person</code> , and <code>pctype</code> from each zip by name (case and folder do not matter). Decodes UTF-8 with a cp1252 fallback, checks that every required column exists, and keeps <code>STATE = 12</code> . Reads the Census file (latin-1) and keeps the 67 county rows. An unreadable member becomes a failed check that names the file.	A member or a required column is missing, or a member cannot be read.
4	reconcile_inventory	3	G3	<code>expected_counts.csv</code> must be present and complete; a missing file fails the gate rather than switching it off. Florida rows per member and year must equal it (for the real files: the Week 6 profile, <code>docs/profile/01_inventory.csv</code>).	The file is not the one that was profiled.

#	Stage	Runs after	Gate	What it does	A failure means
5	transform	4	—	Builds the unit of observation (PER_TYP 5 and INJ_SEV 4), its crashes, their in-transport vehicles, and the typing records. Filler coordinates (77.7777 / 88.8888 / 99.9999) become NULL. A STR_VEH of 0, or one with no vehicle in the same crash, becomes NULL with a warning (rule 2: the death is kept). Census becomes one row per county and year. Every step logs rows in and out.	(no gate)
6	join_block_groups	5	—	Place of the EPA spatial join for crash.geoid10 (cleaning plan, issue 5). Not implemented yet: the stage always logs itself as skipped with a warning, so crash.geoid10 stays NULL.	(no gate)
7	validate_records	5	G4 record rules	Applies pandera schemas, as the proposal planned. Raw-file layer, on every Florida row: every coded cell is a number; keys are unique within the year; no orphans (vehicle, person, and pctype rows need their crash; pctype rows need their person). Record layer: the schema's CHECK ranges, codes with a label valid for that year in code_lookup, county in the 67, PEDCTYPE in pbcate_type, at least one vehicle per crash, one population per county and year. Each failing record goes to rejected_record with file, member, CSV line, key, rule, column, value, and the raw source values.	One or more records rejected. Nothing is published.
8	reconcile_unit	7	G5	Pedestrian fatalities and crashes per year must equal the expected totals (real files: 695 / 819 / 780 / 774 / 668 deaths).	Rows were lost or gained between file and unit.
9	load	6, 8	—	One transaction. Deletes 2020–2024 from crash (cascading to vehicle, pedestrian, ped_crash_type) and from county_population, inserts with COPY, then runs the deferred "one or more" triggers. Logs, per year and table, the rows deleted from the previous load and the rows inserted.	A constraint or trigger refuses a row: rollback.
10	verify_load	9	G6 post-load	Runs inside the same open transaction. Checks that rows per year equal the rows prepared, that no key is duplicated, that pedestrian rows equal the expected totals, that the v_dq_* views return nothing (a failure records the first offending keys), that every crash has a pedestrian and a vehicle, that county rates cover every death, and that the run has five FARS source_file rows. If every blocking check passes, sets etl_run.status = 'succeeded' in the same transaction and commits. Otherwise rolls back.	Rollback; the previous load stays.

#	Stage	Runs after	Gate	What it does	A failure means
11	finalize	10	—	Report only: row counts and an md5 content fingerprint of each data table, and one lineage line per year. An error here cannot undo or mislabel the committed run.	—

Two connections make the failure behaviour visible:

- The **load connection** holds the one transaction of stages 9–10, including the change of the run's status to `succeeded`. Data and status therefore commit together. If a run's row still says `running`, that run never committed data.
- The **bookkeeping connection** runs in autocommit and writes `etl_run`, `source_file`, `etl_log`, `validation_result`, and `rejected_record` as the run goes. The evidence of a failed run therefore survives the rollback of its load.

Each check is one `validation_result` row. Checks marked "warning only" are reported but do not block. There are three: pedestrians without a typing record, pedestrians without a striking vehicle, and the block-group share while the EPA join is pending.

Data lineage

Raw source (pinned)	Member / rows used	Stage that filters it	Target	Real files (Week 6 profile)	Synthetic run 1 (section 2)
FARS{year}NationalCSV.zip	accident.csv, STATE = 12	extract → transform (crashes with a pedestrian death)	crash	15,959 Florida rows → 3,690 crashes	1,304 → 547
same	vehicle.csv, STATE = 12	extract → transform (vehicles of those crashes)	vehicle	25,203 Florida rows → the in-transport vehicles of the 3,690 crashes (count not profiled in Week 6; logged per year by the load)	1,790 → 630

Raw source (pinned)	Member / rows used	Stage that filters it	Target	Real files (Week 6 profile)	Synthetic run 1 (section 2)
same	person.csv , STATE = 12	extract → transform (PER_TYP 5, INJ_SEV 4)	pedestrian	41,133 Florida rows → 3,736 deaths	3,394 → 596
same	pbtype.csv , STATE = 12	extract → transform (records of those deaths)	ped_crash_type	5,193 Florida rows → 3,736	876 → 596
co-est2025-alldata.csv	SUMLEV 050, STATE 12	extract → transform (county × year)	county_population	67 counties × 5 years = 335	67 → 335
(EPA SLD, not yet pinned)	—	join_block_groups (skipped)	block_group , crash.geoid10	—	—

Every successful run records this chain in four places (a failed run records it up to the stage where it stopped):

- `source_file` ties the run to the exact files, by SHA-256 and release.
- `etl_log` holds the rows in and out of every filter.
- Gates G3 and G5 compare the counts with the Week 6 profile.
- `finalize` writes one line per year, from file to loaded rows. For example, synthetic run 1:

```
lineage 2024: FARS2024NationalCSV.zip sha256 6532260bed84 (Annual Report File) -> person 1,114 national -> 63
1 Florida -> 110 pedestrian fatalities loaded
```

2. One successful end-to-end run

Evidence: [docs/evidence/week7_synthetic/01_first_run.log](#) and the full run log [runs/run_0001.log](#) (205 lines).

Run 1 started on an empty database; the fingerprint of the empty tables was `6891cd577154`. It passed all six gates with **93 checks passed, 0 blocking failures, and 1 warning**. The warning is the block-group share of 0%, because the EPA join has not run yet.

`validation_result` stores a warning-only check that does not hold with `passed = false` and the suffix "(warning only)", so `etl.py status` lists it among the failed checks and the history table counts it as `checks_failed = 1`; it does not block. The run committed in under 1.5 seconds on the synthetic files.

Stage	Rows in → out (2020-2024 together)
verify_sources	6 files → 6 verified, 6 <code>source_file</code> rows
extract	accident 2,305 → 1,304 Florida; vehicle 3,245 → 1,790; person 5,850 → 3,394; pbtype 1,331 → 876; census 79 → 67
transform	person 3,394 → 596 pedestrian fatalities; accident 1,304 → 547 crashes (6 filler coordinate pairs set to NULL); vehicle 1,790 → 630; pbtype 876 → 596; census 67 → 335 county-years
validate_records	10,068 rows checked → 0 rejected
reconcile_unit	deaths per year 130 / 120 / 104 / 132 / 110 = expected; crashes 120 / 112 / 94 / 122 / 99 = expected
load	deleted 0; inserted crash 547, vehicle 630, pedestrian 596, <code>ped_crash_type</code> 596, <code>county_population</code> 335
verify_load	25 post-load checks: 24 passed, 1 warning; COMMIT together with status <code>succeeded</code>

Final state, from `python pipeline/etl.py` snapshot:

table		rows		2020		2021		2022		2023		2024		content md5
crash		547		120		112		94		122		99		2e8235061c96
vehicle		630		137		128		104		148		113		8ec4676fa910
pedestrian		596		130		120		104		132		110		52d6d56c153e
ped_crash_type		596		130		120		104		132		110		f58bd3ea79b8
county_population		335		67		67		67		67		67		84cbd56a35e4
fingerprint of all data tables: c0656528b64a4e34ba381d7f356635ef														

The same run reports the provisional-year detector of cleaning-plan issue 2. `RUR_URB` is not reported for 3.6% of the 2024 deaths, against at most 2.3% in 2020–2023 in the synthetic files. The run also keeps the 2024 file labelled `Annual Report File` in `source_file`.

3. Rerun with the same inputs: duplicate prevention

Evidence: [02_rerun_same_inputs.log](#).

Run 2 used the same command and the same files. It passed every gate and replaced each year instead of adding to it:

```
run=2 stage=load INFO rows_in=130 rows_out=130 | 2020 pedestrian: deleted 130 row(s) of the previous load, in
serted 130
...
run=2 stage=finalize INFO | fingerprint of the data tables after the run: c0656528b64a4e34ba381d7f356635ef (i
dential to before the run: the rerun changed nothing)
CHECK PASS content fingerprint identical to scenario 1: c0656528b64a4e34ba381d7f356635ef
CHECK PASS every table and year replaced, not appended: 25 delete/insert lines
```

The snapshots before and after the rerun are identical: the same row counts per table and year, the same md5 of every table's content, and the same combined fingerprint. Four mechanisms make this hold, and each one leaves evidence:

1. **Composite keys.** `crash` (`year`, `st_case`), `vehicle` (`year`, `st_case`, `veh_no`), `pedestrian` and `ped_crash_type` (`year`, `st_case`, `veh_no`, `per_no`), and `county_population` (`county_fips`, `year`). A second copy of a row cannot be stored, and a 2021 case cannot collide with the 2023

case that reuses its number. The Week 5 constraint tests A1–A6

([03_constraint_tests.log](#)) show each key refusing a second copy, and a duplicate that arrives in an input file is stopped even earlier, by G4 (failure B, defect a, in section 4).

2. **Replace by year in one transaction.** The load deletes the five years and inserts them again before anyone can see the result. The log shows "deleted N, inserted N" for every table and year.
3. **G6 checks the result before COMMIT.** Rows per year must equal the rows prepared, `count(*)` must equal `count(DISTINCT key)` in every table, and pedestrian rows per year must equal the expected totals.
4. **Content fingerprint.** `preflight` and `finalize` compute an md5 over every row of the five data tables. Equal fingerprints mean the rerun changed nothing.

The bookkeeping tables grow by design: each run adds its own `etl_run`, `source_file`, `etl_log`, and `validation_result` rows, which is what makes runs comparable.

4. Two deliberate failures and the system's response

Failure A: a raw file edited after it was pinned (gate G1)

Evidence: [03_failure_tampered_file.log](#). The scenario copies the inputs, changes one AGE value in `person.csv` inside the 2022 zip (35 → 47, line 131), and writes the zip again. This is what opening the CSV in a spreadsheet and saving it would do. `sources.csv` is not updated.

```
run=3 stage=verify_sources ERROR | FAIL G1 sha256 FARS2022NationalCSV.zip: expected e543adcea0c609ef... (33, 417 bytes), found 0d82f7da3b4907ca... (33,420 bytes)
run=3 stage=verify_sources ERROR | gate failed at stage verify_sources: G1 integrity: 1 blocking check(s) failed: G1 sha256 FARS2022NationalCSV.zip
run=3 stage=verify_sources INFO | data tables unchanged by this run: True (fingerprint c0656528b64a); they hold the load of run 2
run=3 stage=end ERROR | RUN 3 FAILED at stage verify_sources; see docs/Week7_Pipeline_Shkirpan.md, section 6 (recovery)
```

Response:

- exit code 1, and `etl_run 3` is failed;
- no member was read, and no `source_file` row was written for the run ([07_run_history.log](#): `run 3 failed, files = 0`);
- the data tables are unchanged: the fingerprint is the same as after scenario 1, and the scenario's three checks pass.

Failure B: a new file version with four bad records (gate G4)

Evidence: [04_failure_bad_records.log](#). The scenario writes a new version of the 2023 zip with four defects, one for each kind of problem the cleaning plan says must stop a load. It then accepts the new file's hash and row inventory into a copy of `sources.csv` and `expected_counts.csv`, the way a re-download would be accepted. Gates G1–G3 therefore pass, and the record-level gate has to find the defects:

Injected defect	Reference	Caught by	Rejected record (etl.py rejected, which lists the latest run with rejects: run 4)
(a) exact duplicate of a pedestrian-fatality person row	issue 3 (a duplicate is a pipeline error)	G4 raw file <code>person</code>	<code>person 120002/0/1</code> , <code>person.csv</code> line 124: duplicate key within the year
(b) LGT_COND = 42 on a crash	issue 4 (an unlabelled code stops the load)	G4 record rules <code>crash</code>	<code>crash 2023/120003</code> , <code>accident.csv</code> line 54, <code>lgt_cond = 42</code> : code with a 2023 label in <code>code_lookup</code>
(c) COUNTY = 999 on a crash	business rule 3 (county mandatory and one of the 67)	G4 record rules <code>crash</code>	<code>crash 2023/120004</code> , line 55, <code>county = 999</code> : one of the 67 Florida counties
(d) a pedestrian death and its typing record whose crash is not in <code>accident.csv</code>	issue 3 and business rule 1 (every child row needs its crash)	G4 raw file <code>person</code> , <code>pbtype</code>	<code>person 129999/0/1</code> (line 1212) and <code>pbtype 129999/0/1</code> (line 267): <code>ST_CASE</code> exists in <code>accident.csv</code> (no orphan)

```
run=4 stage=validate_records ERROR rows_in=10074 rows_out=10069 | record-level rules applied to 10,074 rows;
5 record(s) rejected (5 rule failure(s)) into rejected_record
run=4 stage=validate_records ERROR | gate failed at stage validate_records: G4 record rules (5 record(s) in r
ejected_record, run 4): 3 blocking check(s) failed: ...
run=4 stage=validate_records INFO | data tables unchanged by this run: True (fingerprint c0656528b64a); they
hold the load of run 2
CHECK PASS all four defects quarantined (5 records: the orphan death brings its typing record): 5 rejected_r
ecord rows, 4 distinct rules
```

Response:

- exit code 1, and run 4 is `failed`;
- five rows in `rejected_record`: four defects, plus the typing record that the orphan death brings with it;
- the published tables are unchanged.

The orphan death also produced the rule-2 warning in `transform`, because its striking vehicle cannot be resolved. That is the behaviour the plan asks for: a warning, not a silent drop.

Restart after a killed run (scenarios 5 and 6)

Evidence: `05_interrupted_run.log` and `06_restart_after_interrupt.log`. Run 5 was started with `--simulate-crash-after load`. That ends the process with `os._exit(70)` after all rows had been inserted but before COMMIT, as a power cut or a killed terminal would. PostgreSQL rolled the open transaction back when the connection dropped. The snapshot is unchanged, and `etl_run 5` is left `running`. Run 6, the same command again, found it:

```
run=6 stage=preflight WARN | recovered abandoned run 5 (started 2026-10-04T22:26:59-04:00): status set to fai
led; its load transaction was never committed, so no rows of it exist
run=6 stage=end INFO | RUN 6 SUCCEEDED; log file data/interim/week7/logs/run_0006_20261004T222703.log
CHECK PASS tables identical to scenarios 1-2: c0656528b64a4e34ba381d7f356635ef
```

A crash one step later, right after COMMIT (`--simulate-crash-after verify_load`), leaves a consistent state too: the run is already `succeeded`, because its status committed with its data,

so the next run has nothing to recover. This case was checked during development and is not part of the committed evidence.

Other failure paths checked during development (not in the committed evidence)

During the AI-assisted adversarial review described under "Sources and AI assistance" (fixes in commit 83c9bc3), each of these was injected by hand into a copy of the synthetic inputs. As in scenario 4, the changed file's hash and row inventory were accepted into a copy of `sources.csv` and `expected_counts.csv`, so that the defect had to be caught by the gate named rather than by G1 or G3. `run_scenarios.py` does not reproduce these cases.

Defect	Gate
<code>expected_counts.csv</code> missing or incomplete	G3
A member that cannot be decoded	G2 (the check names the file and member)
Blank or non-numeric LATITUDE, STR_VEH, or PER_TYP	G4 raw file (<code>blank or not a number</code>)
A typing record with no person row	G4 raw file
A crash whose vehicle rows are missing	G4 record rules
Two Census rows for one county	G4 record rules
FATALS smaller than the crash's pedestrian rows	G6 <code>v_dq_count_mismatch</code> , rolled back; the detail names the crash

A trailing delimiter on every CSV line is read correctly.

5. Logs: run ID, timestamps, row counts, errors

Every run leaves the same evidence in five places:

Where	What one record holds	How to read it
<code>logs/run_NNNN_<start time>.log</code> for <code>etl.py</code> run (the default <code>--log-dir</code>); the scenario runner writes to <code>data/interim/week7/logs/</code> . Both are ignored by Git; the cited runs are copied to <code>runs/run_NNNN.log</code>	ISO timestamp with UTC offset and milliseconds, <code>run=</code> , <code>stage=</code> , <code>level</code> , <code>rows_in=</code> / <code>rows_out=</code> where rows move, message	any text editor; <code>grep "ERROR\ WARN"</code> <code>docs/evidence/week7_synthetic/runs/run_0004.log</code>
<code>etl_run</code>	run ID, start, finish, git commit, status running / succeeded / failed	<code>python pipeline/etl.py status</code>

Where	What one record holds	How to read it
etl_log	run ID, step, rows in, rows out, ok / warning / error, message, logged_at	python pipeline/etl.py status --run N
validation_result	run ID, check name with its gate (G1-G6), passed, detail with expected and actual values, checked_at	the same command lists the failed checks
rejected_record	run ID, year, file, member, CSV line, target table, key, rule, column, failing value, raw source values (JSON), rejected_at; one row per record and broken rule	python pipeline/etl.py rejected --run N

Excerpts from the synthetic runs (full files under docs/evidence/week7_synthetic/):

```
# a row-count line (stage transform, run 1)
2026-10-04T22:26:47.227-04:00 run=1 stage=transform INFO rows_in=718 rows_out=130 | 2020 person -> pedestria
n: PER_TYP = 5 and INJ_SEV = 4

# a gate passing, with expected and actual values (run 1)
2026-10-04T22:26:48.106-04:00 run=1 stage=reconcile_unit INFO | PASS G5 2024 ped_fatalities: expected (Week
6 profile / published) 110, derived 110

# an error with the record that caused it (run 4)
2026-10-04T22:26:57.151-04:00 run=4 stage=validate_records ERROR | REJECT crash 2023/120004 (FARS2023National
CSV.zip:accident.csv line 55): one of the 67 Florida counties [county=999]

# the run outcome (run 3)
2026-10-04T22:26:54.217-04:00 run=3 stage=end ERROR | RUN 3 FAILED at stage verify_sources; see docs/Week7_Pi
pline_Shkirpan.md, section 6 (recovery)
```

07_run_history.log holds the history of all six runs, read back from the database:

- runs 1, 2, and 6 succeeded with 93 checks passed each, plus the one warning-only check (counted as checks_failed = 1, see section 2);
- run 3 failed at G1, run 4 at G4, and run 5 was killed and then closed by run 6;
- then one table of every run's error, warning, load and finalize lines, in log order, with their row counts;
- then all 19 scenario checks.

The synthetic run was recorded in the America/New_York time zone (−04:00, Florida's). Log lines carry the offset and milliseconds. `etl.py status` prints database times in the same zone but rounded to whole seconds and without the offset, so a start logged at 22:27:03.744 can show as 22:27:04.

6. Restart, recovery, and rejected-record instructions

6.1 Normal run and rerun

```
python pipeline/etl.py run          # defaults: data/raw/, pipeline/sources.csv, pipeline/expected_counts.csv, logs/
python pipeline/etl.py status      # did it succeed? which stage failed? which checks?
```

A rerun is always safe. It either replaces 2020–2024 completely, after passing every gate, or it changes nothing. There is no partial state to clean up and no "resume from stage N" step: to restart, run the same command again. The whole run takes about 1.5 seconds on the synthetic files. It has not been timed on the real files; the Week 6 profile read the same zips in about 30 seconds.

The database is chosen with `PGDATABASE` (default `ped_safety`) or `--dsn`. Without the real files, the same commands work on the synthetic ones:

```
python tools/make_synthetic_raw.py          # writes data/synthetic/clean/
python pipeline/etl.py run --raw-dir data/synthetic/clean --sources data/synthetic/clean/sources.csv --expected data/synthetic/clean/expected_counts.csv
```

Running the second command again shows a rerun replacing the rows.

Exit code	Meaning	What to do
0	succeeded; tables replaced and verified (a warning after COMMIT can only mean a report line is missing)	nothing
1	a gate failed or an error occurred before COMMIT; tables unchanged	<code>etl.py status</code> , then 6.3
2	could not start (database unreachable or not built)	check the PG* variables; build a new database with <code>sql/00_build_all.sql</code>
3	another run holds the lock	wait for it; never run two at once
70	only with <code>--simulate-crash-after</code> (a test)	run again

6.2 Diagnose a run

1. `python pipeline/etl.py status --run N` shows the `etl_run` row, every `etl_log` step with rows in and out, the failed checks with their details, and the number of rejected records.

2. `logs/run_NNNN_*.log` holds the same steps with millisecond timestamps, plus the traceback of any unexpected error.
3. `python pipeline/etl.py rejected --run N` lists the quarantined records.
4. `python pipeline/etl.py snapshot` shows row counts and fingerprints now. Compare them with the `finalize` lines of the last run that succeeded.

6.3 Recovery by failure

Failed at	Typical cause	Recovery
G1 <code>verify_sources</code>	a raw file was changed, replaced, truncated, or is missing	Never edit a raw file to make it pass. Download it again from the URL in <code>sources.csv</code> (or restore your own unmodified copy). Check that <code>sha256sum</code> equals the pinned value, then rerun. If NHTSA really published a new version, follow 6.5.
G2 <code>extract</code>	NHTSA renamed or dropped a column, a member is missing, or a member cannot be decoded	Confirm in the FARS manual. Map the new name in <code>REQUIRED</code> in <code>etl.py</code> in a reviewed commit, or download the file again; then rerun.
G3 <code>reconcile_inventory</code>	<code>expected_counts.csv</code> is missing or incomplete, or the file is not the one profiled (a different release)	Restore <code>expected_counts.csv</code> from Git. For a new release, run <code>pipeline/profile_raw.py</code> on it and compare with <code>docs/profile/</code> ; only then update the expected counts (6.5).
G4 <code>validate_records</code>	bad or unexpected records	Follow 6.4.
G5 <code>reconcile_unit</code>	deaths lost or gained between file and unit	Compare the transform lines of <code>etl_log</code> with the profile. A difference is a code defect; fix it before rerunning.
G6 <code>verify_load</code> , or a constraint error in <code>load</code>	the load would break a rule the earlier gates did not see	The transaction was rolled back. For a failed check, read its full detail with <code>SELECT check_name, detail FROM validation_result WHERE run_id = N AND NOT passed;</code> (<code>status</code> shortens details to 100 characters); a failed <code>v_dq_*</code> check lists its first offending rows. A constraint error has no check row: read its error line in <code>etl_log</code> or in the run log. Fix the rule or the data source, then rerun.
process killed, machine restarted, connection lost	—	Run the same command. The next run closes the abandoned <code>etl_run</code> row as <code>failed</code> . Its load was never committed: a run becomes <code>succeeded</code> only in the same transaction as its data.
exit 3 (lock held)	a run is in progress, or a crashed session still holds its connection	Wait. If no run is active, find the holder with <code>SELECT a.pid, a.state, a.query_start FROM pg_locks l JOIN pg_stat_activity a USING (pid) WHERE l.locktype = 'advisory';</code> The lock disappears when that connection ends (<code>SELECT pg_terminate_backend(<pid>);</code> ends it).

To roll back to an earlier good load, rerun with the earlier inputs. Check out the `sources.csv` and `expected_counts.csv` of that commit, keep the matching raw files, and run. The load replaces 2020–2024, so no manual delete is needed.

6.4 Rejected records

1. **Read them.** `python pipeline/etl.py rejected --run N` lists file, member, CSV line, key, rule, column, and value. `SELECT raw_record FROM rejected_record WHERE run_id = N` shows the source values as read. Nothing in a raw file is ever edited, and nothing was published.
2. **Decide what each record is.** Every rejection is one of three cases:
3. **The record is right and the rule is incomplete.** An example is a new FARS code documented in the current Analytical User's Manual. Add the label to `sql/07_seed_codes.sql` (with `year_from`) so that a future rebuild has it, and add the same row to the live database with an `INSERT INTO code_lookup ...` in a reviewed migration file (for example `sql/migrations/2026-10-12_new_code.sql`, applied with `psql -d ped_safety -v ON_ERROR_STOP=1 -f <file>`). If the column's CHECK must change, do it with `ALTER TABLE ... DROP CONSTRAINT ..., ADD CONSTRAINT ...` in the same file. Commit, apply the migration, and rerun. Do not rebuild the database for this: a rebuild would erase the run history, including the rejected records.
4. **The file is wrong.** Examples are duplicates, orphans, and a truncated or corrupted download. Download it again, verify the checksum, and rerun. Report a defect in an official file to NHTSA rather than patching it.
5. **The record is real but outside the rules on purpose.** An example is COUNTY 999 (unknown). The plan keeps every death in a county rate, so this needs a written decision, recorded in the cleaning plan before the rule changes: either an `unknown` county reference row, or an exclusion with a count in the report. It is never dropped silently.
6. **Rerun.** When the rerun passes G4, the death is loaded with all the others. The old `rejected_record` rows stay as the history of the failed run. They are removed only together with their `etl_run` row (`ON DELETE CASCADE`).
7. **Privacy.** `rejected_record.raw_record`, `record_key`, and `failure_value` hold case-level values. The data dictionary marks them Restricted, and they stay in the local database. Evidence published from real files uses `--redact-keys`. It masks case numbers in every log line and hides the key, CSV line, and value of each rejected record. `run_scenarios.py` also leaves the original values out of the descriptions of the injected defects.

The gate tolerates **zero** rejected records. The raw files profiled in Week 6 contain no duplicates or orphans, so a rejected record signals that the input or the rules changed. Loading the rest would quietly change the count of deaths, which is the unit of the project.

6.5 Accepting a new source version (the 2024 Final file)

1. Download it to `data/raw/` under a new name and make it read-only. Keep the Annual Report File until the end of the project.
2. Run `python pipeline/profile_raw.py` against it and review the differences from the profile (cleaning plan, issue 2).
3. Update the 2024 row in `pipeline/sources.csv` (file name, bytes, SHA-256, `release = Final`, retrieval date) and the 2024 rows of `pipeline/expected_counts.csv`, then commit.
4. Run `python pipeline/etl.py run`. The 2024 rows are replaced, not added. The load log shows the old and new 2024 row counts, `source_file` records the new release, and G6

confirms that the 2020–2023 counts still equal their expected totals.

7. Completion criteria

Criterion	Where it is shown
The pipeline can be rerun safely	Section 3 (identical fingerprints after a rerun), scenario 6 (restart after a killed run), the atomic status in section 1, section 6.1
Validation gates and failure behaviour are visible	Section 1 (six gates), section 4 (G1 and G4 failures with the database unchanged, other failure paths), <code>validation_result</code> and <code>rejected_record</code> , the CHECK lines of every scenario
Logs contain enough evidence to diagnose a run	Section 5: run ID, timestamp with offset, stage, rows in and out, the failing check with expected and actual values, the rejected record with file and line
Recovery steps are explicit	Section 6: exit codes, diagnosis steps, a recovery table per gate, the rejected-record procedure, the new-version procedure
Results connect to the documented data lineage	Section 1 lineage table; <code>source_file</code> (SHA-256 and release) per run; G3 and G5 against the Week 6 profile counts; the per-year lineage lines in <code>finalize</code>

8. Methods basis

The validation stages implement the data-quality concepts of the course text (Tan, Steinbach, Karpatne, & Kumar, 2019, chapter 2, "Data"):

- **Measurement and data-collection errors** are checked where they can enter: an altered or incomplete file (G1, G3), missing attributes (G2), and values that are blank, non-numeric, or outside an attribute's domain (G4).
- **Missing values** are handled as decided in Week 6. FARS unknown codes stay as values, and only the documented coordinate fillers become NULL.
- **Inconsistent values** are found by comparing facts recorded twice: FATALS against the pedestrian rows, the county code against the block group, and the expected totals against the derived unit (G5).
- **Duplicate data** is tested on keys. Week 6 established that look-alike rows (multi-victim crashes, reused case numbers) are distinct objects, so the pipeline never deduplicates.
- **Timeliness** is the problem of the provisional 2024 file. The release label and the replace-by-year load handle it.

The aggregation and discretization that prepare the transactions for association analysis (chapters 5–6) stay in the view `v_ped_transactions`. The loaded data therefore keep the codes exactly as published.

9. Limitations and next steps

- **The committed logs are from synthetic files.** The pipeline, the checksums, and the expected counts for the real files are in place. With the real zips in `data/raw/`, python

`pipeline/run_scenarios.py` produces the same evidence in `docs/evidence/week7/`, with the real counts (3,736 deaths). A real-data run can still find a code that the seeds do not label. G4 then stops the load, and 6.4 applies.

- **The EPA block-group join is not implemented yet.** The stage has its place in the order and logs itself as skipped. Until the EPA file is pinned, `crash.geoid10` is NULL and the walkability item reads `unknown`, and the block-group share check is a warning, not a gate.
- **The comparison of the 2024 Annual Report File with the Final file is partial.** The load logs the old and new row counts per year and table. The per-variable count of changed codes that the cleaning plan promises (issue 2, action e) will be added with the Final file.
- **Single machine, single user.** The advisory lock prevents concurrent runs on one database. There is no scheduler, because the sources change at most once a year.

Sources and AI assistance

- Tan, P.-N., Steinbach, M., Karpatne, A., & Kumar, V. (2019). *Introduction to data mining* (2nd ed.). Pearson. Chapter 2 (data quality and preprocessing); chapters 5–6 (association analysis).
- National Highway Traffic Safety Administration. (2026). *Fatality Analysis Reporting System (FARS): National CSV files 2020–2024*. <https://static.nhtsa.gov/nhtsa/downloads/FARS/>
- National Highway Traffic Safety Administration. (2026). *FARS analytical user's manual, 1975–2024*. <https://static.nhtsa.gov/nhtsa/downloads/FARS/Links%20for%20FARS%20Manuals.pdf>
- U.S. Census Bureau. (2026). *County population totals: 2020–2025, Vintage 2025* (CO-EST2025-ALLDATA). <https://www.census.gov/data/tables/time-series/demo/popest/2020s-counties-total.html>
- pandera (data validation for pandas), psycopg 3, and the PostgreSQL 16 documentation on transactions and advisory locks.

I used Claude (Anthropic), an AI assistant, to help write the pipeline, the scenario runner, the synthetic-data generator, and the wording of this document, and to run an adversarial review of the pipeline whose findings were fixed before the evidence was recorded. The logs in `docs/evidence/week7_synthetic/` come from a run on PostgreSQL 16.14 (recorded at the top of `01_first_run.log`) in the assistant's Linux environment, on synthetic files only. The question, scope, rules, and design decisions come from my approved proposal and my Week 4–6 work. I reviewed the generated material and I am responsible for the content.